

Recetas Deliciosas

Como Servicio

Guevara Badillo Areli Alejandra
Ocaña Castro Héctor Rigel





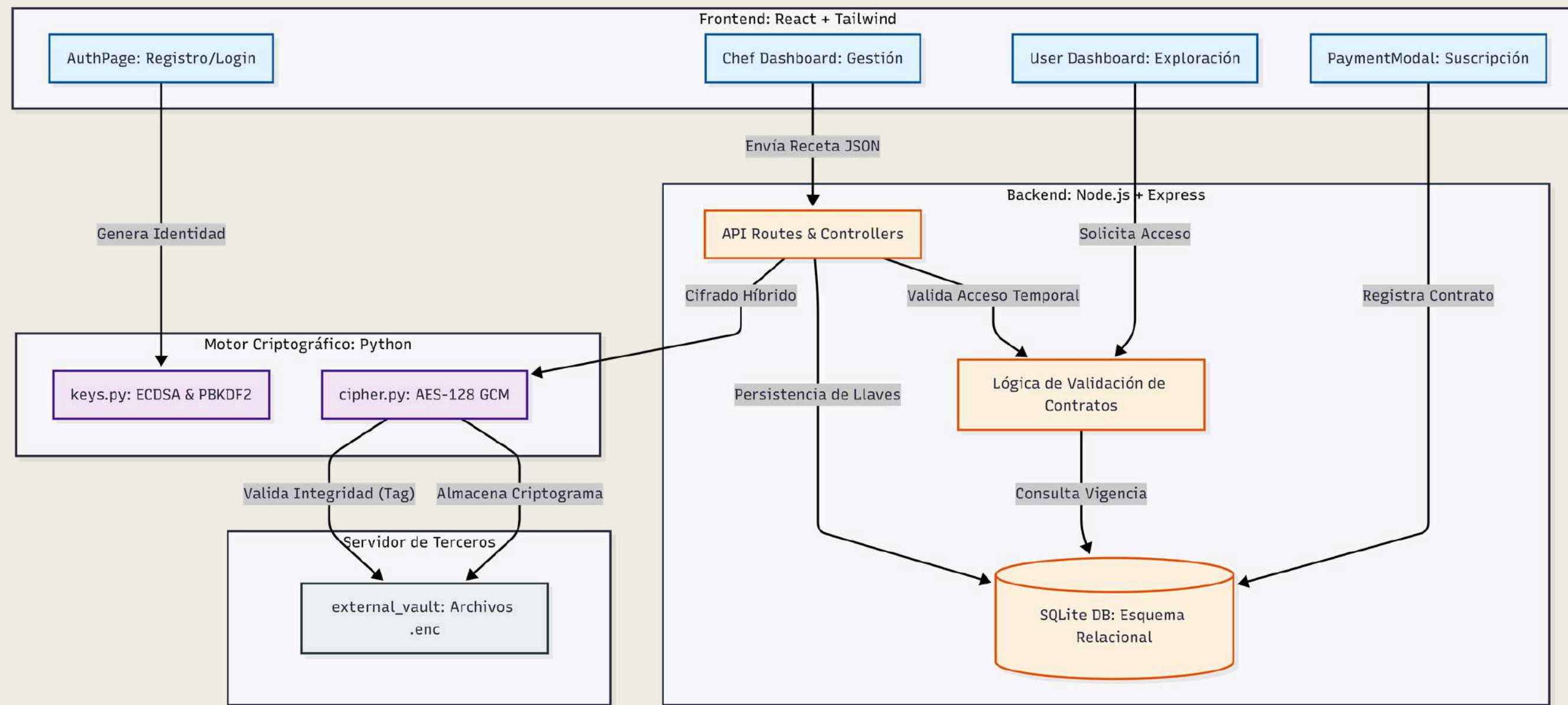
El Contexto

"Una chef mexicana quiere obtener unos beneficios extra con sus recetas. Ha decidido ofrecer un servicio web para compartir sus recetas. Ha decidido que cualquier persona que desee tener acceso a sus recetas debe pagar una suscripción. Los suscriptores deben firmar un contrato con la chef. Este contrato establece el coste y el periodo (3, 6 o 12 meses) de la suscripción. Las recetas se almacenan en un servidor de un tercero, y la chef quiere garantizar la confidencialidad de sus recetas, de modo que solo ella y sus suscriptores tengan acceso a ellas. Además, quiere utilizar criptografía de clave secreta para garantizar que, una vez finalizada la suscripción, las personas no tengan acceso a las recetas. La chef quiere realizar el proceso descrito anteriormente sin papel y utilizando criptografía híbrida. Imagina que la chef te ha contratado a ti y a tu equipo para proporcionar una solución"



LISTA DE AVANCES

Actividad	Descripción	Responsable
Diseño de la Base de Datos	Creación del esquema relacional en SQLite para gestionar usuarios, chef, recetas y la persistencia de contratos y claves.	Areli Guevara
Criptografía de Identidad (keys.py)	Implementación de generación de pares de claves ECDSA y protección de la clave privada, y logica para su aplicación en la página de autenticación de usuarios.	Rigel Ocaña
Criptografía de Contenido (cipher.py)	Desarrollo del motor de cifrado híbrido utilizando AES-128 GCM para garantizar confidencialidad e integridad de las recetas, y logica en la creación y guardado de las recetas del lado del dashboard del chef.	Areli Guevara
Validaciones y Control de Acceso	Implementación de validaciones y lógica de verificación de contratos para denegar el acceso a recetas si la suscripción ha expirado.	Rigel Ocaña
Diseño Frontend	Diseño visual de la página de autenticación y los dashboard de chef y usuarios.	Areli Guevara
Funcionalidades Dashboar del Subscriptor	Implementaciones de funciones básicas en el dashboar del subscriptor (simulación de pago, creación de contratos, favoritos y suscripciones).	Rigel Ocaña



Arquitectura

- 1. Capa de Cliente (Frontend):** Es donde la Chef publica y el suscriptor consume. El PaymentModal es el disparador que crea el derecho de acceso en la base de datos.
- 2. Capa de Servidor (Backend):** Actúa como el orquestador. Mueve los datos y contiene la Lógica de Validación de Contratos.
- 3. Motor criptográfico Python (/crypto_vault):** Es el "corazón" de seguridad.
 - a. keys.py: genera pares de claves ECDSA P-256, deriva claves AES a través de PBKDF2 y cifra/descifra claves privadas con AES-128-GCM
 - b. cipher.py: cifra y descifra el contenido de la receta utilizando AES-128-GCM con una clave simétrica por receta.
- 4. Servidor de Terceros:** Representa el requisito de la Chef de usar un almacenamiento externo. Este servidor solo ve archivos .enc (texto cifrado), cumpliendo con la confidencialidad total.

BASE DE DATOS

Identidad Digital (usuarios / chef):

- clave_publica: Identidad asimétrica para intercambio de secretos.
- clave_privada_cifrada: Protegida con PBKDF2.

Gestión de Contratos (contrato):

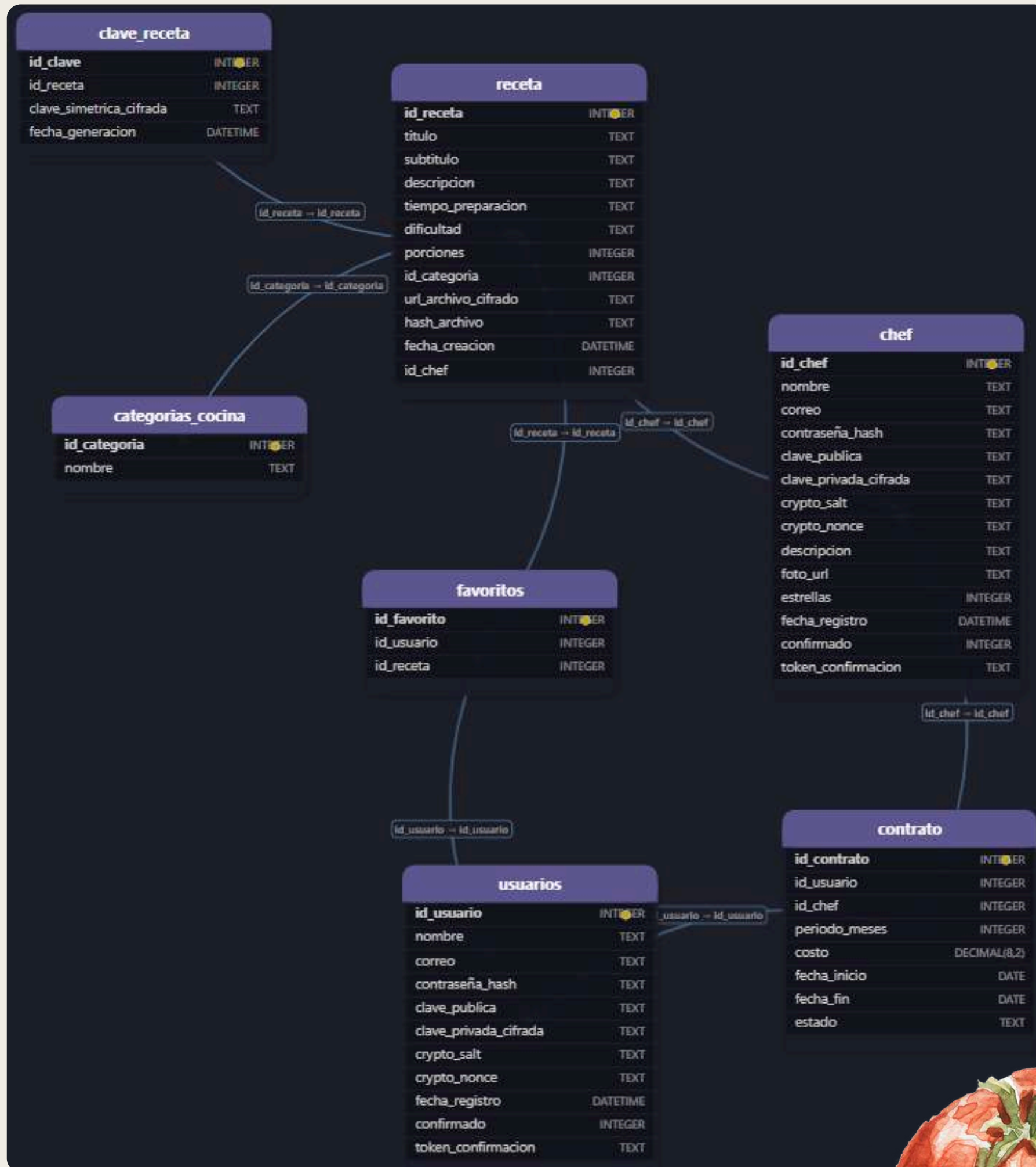
- fecha_fin: Parámetro crítico para la revocación automática de acceso.
- Restricción de periodos: 3, 6 o 12 meses.

Bóveda de Recetas (receta):

- url_archivo_cifrado: Cumple el requisito de confidencialidad en servidores de terceros.
- hash_archivo: Verifica la integridad del contenido externo.

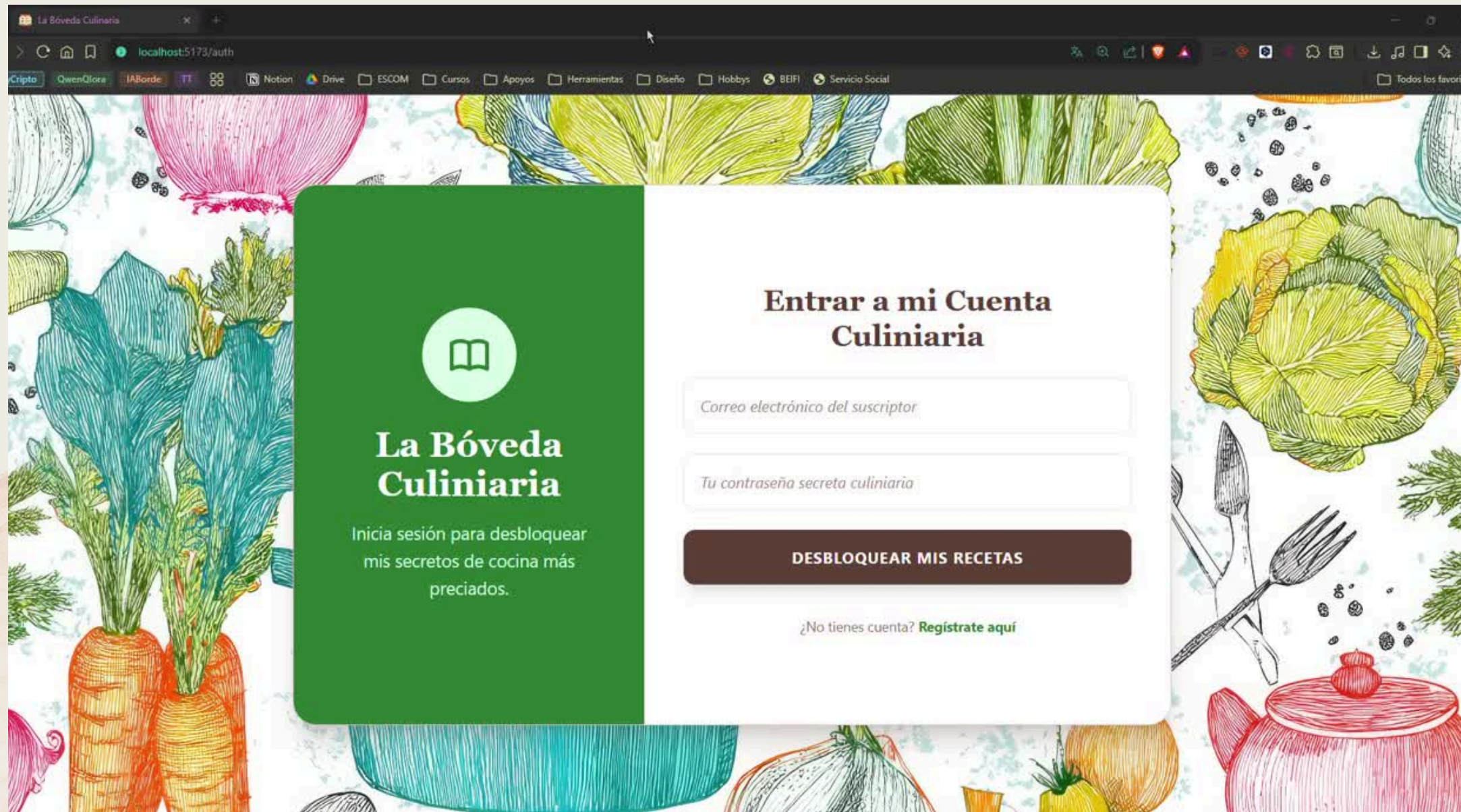
Acceso Condicional (clave_receta):

- clave_simetrica_cifrada: Almacena la llave AES única de la receta.
- Seguridad: El sistema solo entrega esta llave si existe un contrato vigente.



Lógica de Autenticación

AuthPage.jsx



Registro de Nuevas Cuentas:

1. Envas tus credenciales (nombre, correo electrónico y contraseña).
2. El backend genera un par de claves.
3. Se inserta todo en el `usuarios` (o `chef`) tabla.
4. Se envía un enlace de confirmación al correo.

Tanto los chefs como los suscriptores pasan por lo mismo: generación de clave ECDSA P-256, cifrado de clave privada basado en contraseña (PBKDF2 + AES-128-GCM) y confirmación por correo a través de un token aleatorio.

Inicio de Sesión:

1. Envías tu correo electrónico y contraseña.
2. El backend busca su registro en `usuarios` o `chef` y verifica su contraseña.
3. El backend vuelve a derivar la clave AES de su contraseña y sal usando PBKDF2, luego descifra el texto cifrado de la clave privada con AES. Un descifrado exitoso demuestra que su contraseña es correcta y que su material clave está intacto.
4. La API devuelve su ID de usuario, nombre y rol (usuario o chef) y redirecciona a su dashboard correspondiente.

Detras de la Generación de Llaves

Modelo parcial de conocimiento cero keys.py

Generación de pares de claves ECDSA P-256

Al registrarse, la plataforma llama `crypto_vault/keys.py` para generar un nuevo par de claves utilizando la curva elíptica P-256.

La clave publica se almacena en la base de datos en formato PEM, y la clave privada se cifra.

Flujo de protección clave

La clave privada se cifra inmediatamente, deriva una clave AES de la contraseña usando PBKDF2 y luego cifra la clave privada con AES-128-GCM.

Descifrado de claves al iniciar sesión

Al iniciar sesión, la plataforma vuelve a derivar la misma clave AES usando su contraseña y la sal almacenada, luego descifra la clave privada.

```
# Generación de llaves
private_key = ec.generate_private_key(ec.SECP256R1())

public_pem = private_key.public_key().public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
).decode('utf-8')

private_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.TraditionalOpenSSL,
    encryption_algorithm=serialization.NoEncryption()
)

# Flujo de protección de llaves
salt = os.urandom(16)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=16, salt=salt, iterations=100000)
aes_key = kdf.derive(password_str.encode())

aesgcm = AESGCM(aes_key)
nonce = os.urandom(12)
encrypted_private = aesgcm.encrypt(nonce, private_pem, None)

# Descifrado de claves al iniciar sesión
def decrypt_private_key(password_str, encrypted_hex, salt_hex, nonce_hex):
    try:
        salt = bytes.fromhex(salt_hex)
        nonce = bytes.fromhex(nonce_hex)
        encrypted_data = bytes.fromhex(encrypted_hex)

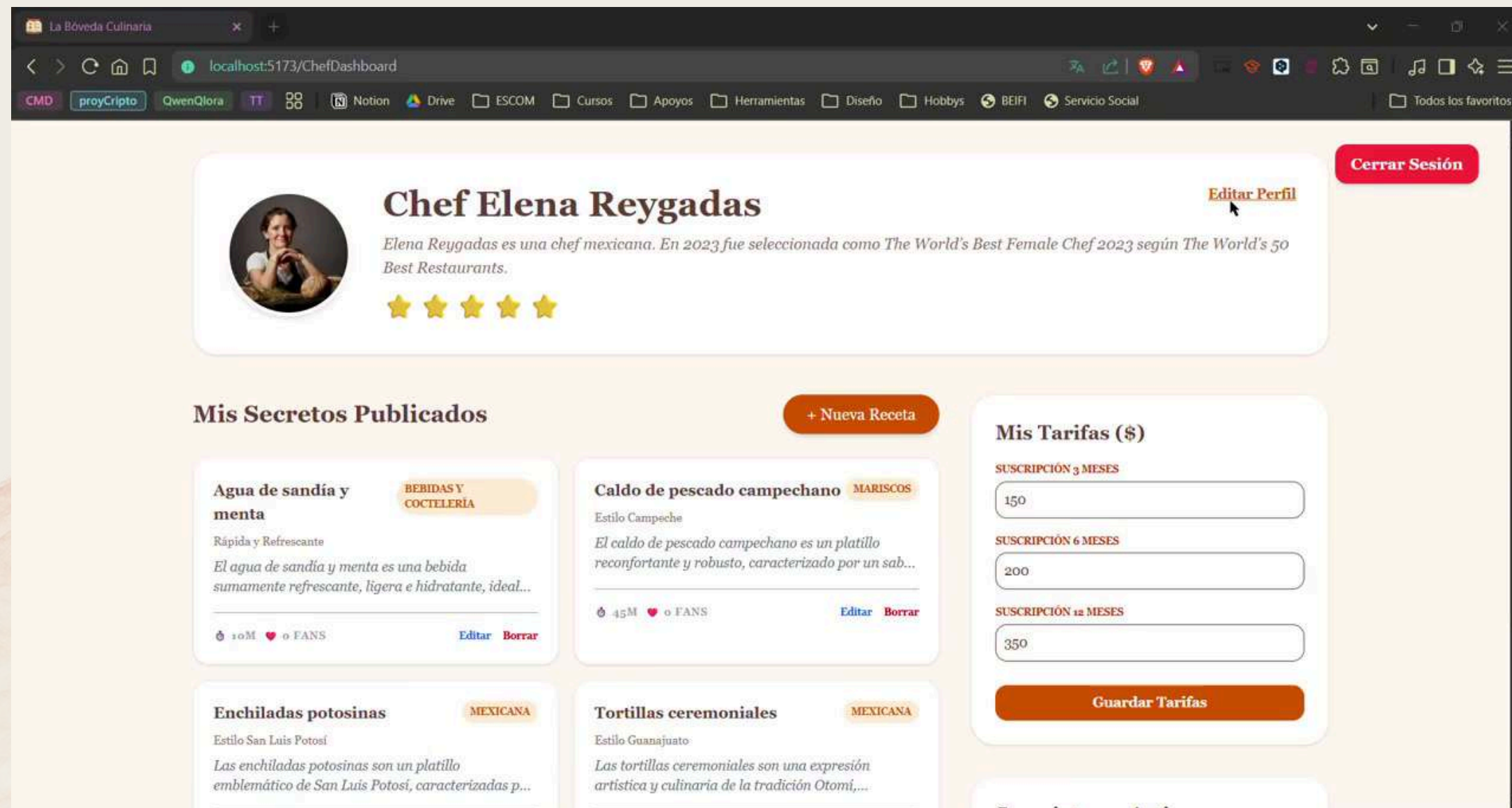
        kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=16, salt=salt, iterations=100000)
        aes_key = kdf.derive(password_str.encode())

        aesgcm = AESGCM(aes_key)
        decrypted_pem = aesgcm.decrypt(nonce, encrypted_data, None)

        return {"status": "success", "private_key": decrypted_pem.decode('utf-8')}
    except Exception:
        return {"status": "error", "message": "Contraseña incorrecta para descifrar llave"}
```

Bóveda del Chef

ChefDashboard.jsx



Subir una receta.

1. Se completan los detalles descriptivos (Título, Dificultad, Categoría, etc.). Esta información se almacena como texto plano en la base de datos.
2. Se ingresan los ingredientes, pasos de preparación y galerías de imágenes.
3. Al dar clic en Cifrar y Publicar se cifra la lista de ingredientes, los pasos de preparación y las imágenes, e guarda en un archivo cifrado .enc en la bóveda externa, se registran los metadatos de la receta y a Hash de integridad SHA-256 del texto cifrado en la base de datos y se almacena la clave AES asociada.

Editar una receta

1. La plataforma recupera el .enc archivo de la bóveda, lo descifra utilizando la clave AES almacenada y completa el formulario de receta con su contenido existente.
2. Realice cualquier cambio en la receta y al guardarla la plataforma genera una nueva clave AES, vuelve a cifrar el contenido, escribe uno nuevo .enc archivo a la bóveda y elimina el archivo cifrado antiguo.

Cifrado de las Recetas

cipher.py

Flujo de cifrado de carga de recetas

Cuando carga una receta, el servidor genera una clave AES nueva de 128 bits, se genera un nuevo nonce de 12 bytes para cada llamada de cifrado que se utiliza `os.urandom` y se cifra el archivo con AES-128-GCM.

Se calcula un hash SHA-256 del texto cifrado sin procesar y lo almacena en `receta.hash_archivo`.

Cifrado autenticado AES-GCM

La etiqueta incorporada de AES-GCM detecta la manipulación a nivel de bytes dentro de un archivo. El hash SHA-256 detecta el reemplazo a nivel de archivo.

```
# Flujo de cifrado de carga de recetas
def encrypt_content(data_json):
    key = os.urandom(16)
    aesgcm = AESGCM(key)
    nonce = os.urandom(12)

    ciphertext = aesgcm.encrypt(nonce, data_json.encode('utf-8'), None)
    sha256_hash = hashlib.sha256(ciphertext).hexdigest()

    return {
        "key": key.hex(),
        "nonce": nonce.hex(),
        "ciphertext": ciphertext.hex(),
        "hash": sha256_hash
    }

# Cifrado autenticado AES-GCM
def decrypt_content(nonce_hex, ciphertext_hex, key_hex, hash_esperado):
    ciphertext = bytes.fromhex(ciphertext_hex)
    hash_calculado = hashlib.sha256(ciphertext).hexdigest()

    if hash_calculado != hash_esperado:
        raise ValueError("El hash del archivo no coincide con el registro oficial.")

    key = bytes.fromhex(key_hex)
    nonce = bytes.fromhex(nonce_hex)
    aesgcm = AESGCM(key)

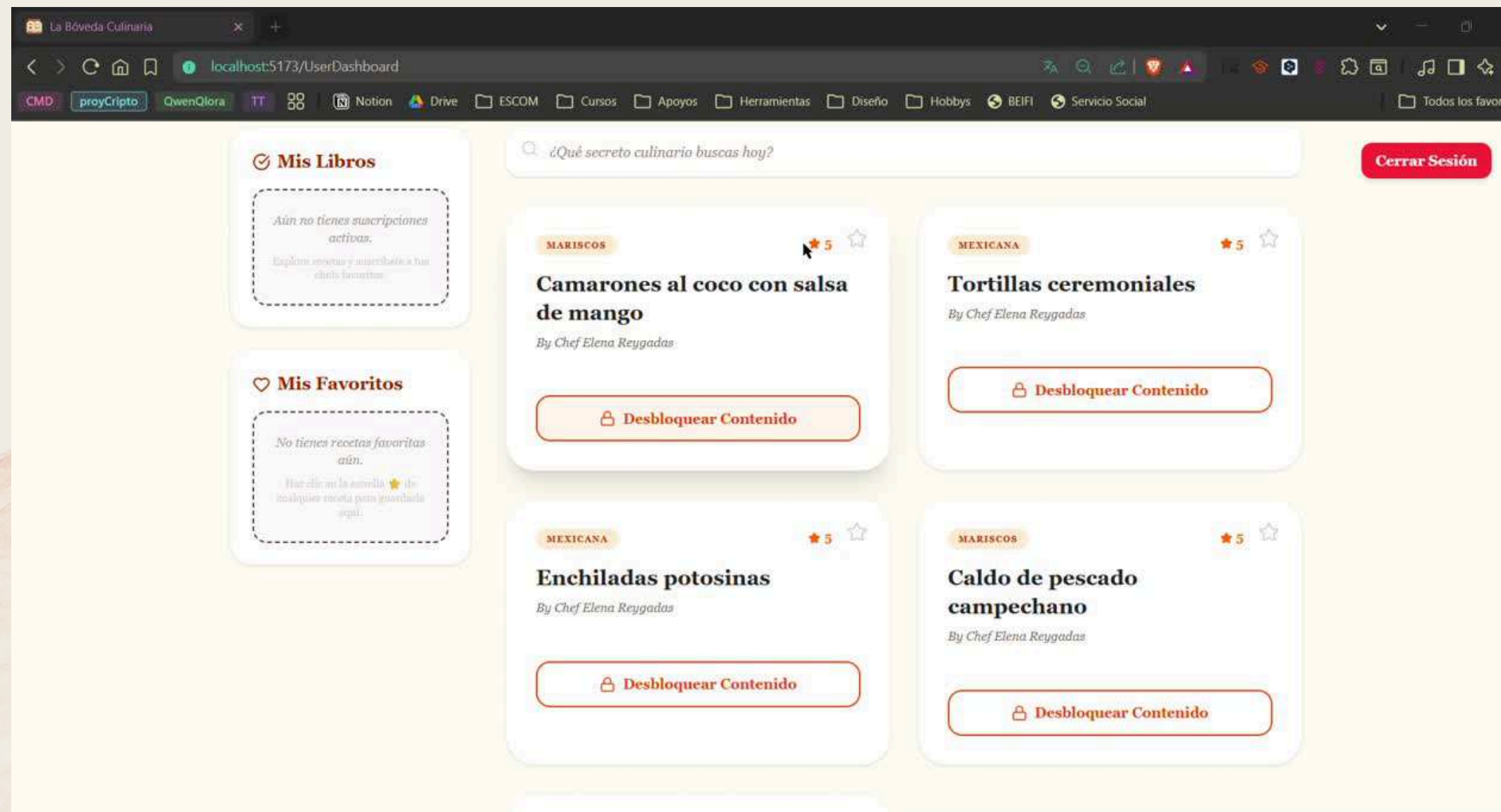
    decrypted_data = aesgcm.decrypt(nonce, ciphertext, None)
    return decrypted_data.decode('utf-8')
```


Cocina del Subscriptor

UserDashboard.jsx



Una suscripción de chef le brinda acceso completo a todas las recetas que el chef ha publicado en la bóveda. Se elige la duración del plan al finalizar la compra y el acceso se otorga inmediatamente y permanece activo hasta la fecha de finalización del plan.



1. **Ver suscripciones activas** — consultar los contratos vigentes con chefs, incluyendo fechas de expiración y costos.
2. **Suscribirse a un chef** — crear contratos de 3, 6 o 12 meses que desbloqueen todas las recetas de ese chef.
3. **Acceder a recetas** — descifrar y leer el contenido completo (ingredientes y pasos) de recetas de chefs a los que está suscrito (*Pendiente*).
4. **Gestionar favoritos** — agregar o quitar recetas de su lista de favoritos.

Al suscribirse el usuario puede ver los datos del chef y tener acceso a todas sus recetas.

Acceso a las Recetas

Creación del Contrato (Suscripción)

Cuando un usuario decide suscribirse, el sistema crea un "Contrato" de acceso. El tiempo de expiración se calcula estrictamente en el servidor, evitando que un usuario malintencionado manipule las fechas desde su navegador.

Límite de Acceso por Tiempo

Cada vez que un suscriptor intenta abrir una receta, el controlador *getRecipeContent* verifica en la base de datos que el usuario tenga un contrato, que el contrato no haya sido cancelado, y que el día no supere la fecha de fin.

```
// Creación del contrato
const fecha_inicio = new Date();
const fecha_fin = new Date();
fecha_fin.setMonth(fecha_fin.getMonth() + parseInt(periodo_meses));

await pool.query(
  `INSERT INTO contrato
  (id_usuario, id_chef, periodo_meses, costo, fecha_inicio, fecha_fin, estado)
  VALUES (?, ?, ?, ?, ?, ?, 'activo')`,
  [id_usuario, id_chef, periodo_meses, costo,
  fecha_inicio.toISOString().split('T')[0],
  fecha_fin.toISOString().split('T')[0]]
);

// Limite de acceso por tiempo
const contrato = await pool.query(
  `SELECT * FROM contrato
  WHERE id_usuario = ? AND id_chef = ?
  AND estado = 'activo'
  AND fecha_fin ≥ DATE('now')`,
  [id_usuario, id_chef]
);

if (!contrato.length) {
  return res.status(403).json({
    status: 'expired',
    message: 'Acceso denegado. Tu suscripción a este Chef ha expirado o no existe.'
  });
}
```




Mecanismo de Acceso a la Receta

en proceso...



ENTREGA DEL PAQUETE CIFRADO:

Una vez validado el acceso, el servidor extrae el archivo de la bóveda (external_vault) y lo entrega al suscriptor.

INTERCAMBIO DE LLAVES

El sistema evolucionará a un modelo de Confianza Cero mediante llaves ECDSA. En lugar de que el servidor descifre el contenido, la llave simétrica de la receta se compartirá cifrada utilizando la llave pública ECDSA del suscriptor.

DESCIFRADO LOCAL Y PRIVADO

Para ver la receta, el dispositivo del suscriptor utilizará su propia llave privada ECDSA para revelar la llave simétrica y, finalmente, desbloquear la receta de manera local. Esto garantiza un Cifrado de Extremo a Extremo (E2EE) donde ni siquiera el servidor puede leer el contenido.





Gracias

Por la Atención

